FEWLLM Data Agent

KDD Cup 2026 DataAgent-Bench — Course Project

MA Ruixin, LIN Yipeng, XIAO Yao

Society Hub

The Hong Kong University of Science and Technology (Guangzhou)

Spring 2026

香港科技大學(廣州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Outline

Task & Scoring Rule

Input: `task.json` + heterogeneous context
(CSV, SQLite, JSON, Markdown rules, record-text prose, images)

Output: `prediction.csv` scored by **column-content signature matching**

- Column names **ignored**; row order **ignored**
- Multi-column match via bipartite augmenting paths
- Normalize: numeric $\to 10^{-2}$, string `strip+lower`

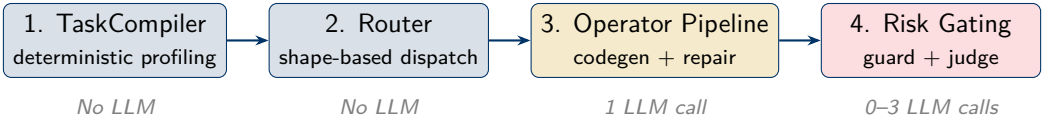
$$\text{score} = \max\left(0,\ \text{recall} - \lambda \cdot \frac{\text{extra\_cols}}{\text{pred\_cols}}\right), \quad \lambda = 0.5$$

Design implications

- **Fewer columns** — every extra wrong col is penalized
- **Preserve granularity** — `first_name+last_name` beats merged `name`
- **Match dtypes** — `163109.0` $\neq$ `163109`

Why Layered? — Four-Layer Architecture

Vanilla ReAct breaks down: easy tasks waste tokens on multi-turn tool calls; hard tasks drift in intermediate steps; no structural guarantee on output shape; no cheap way to verify semantic correctness.



Design principle

Simple tasks pay minimal LLM cost; only suspicious answers escalate to expensive semantic verification.

TaskCompiler & Router — Dispatch by Shape, Not Difficulty

TaskCompiler (zero LLM):

- Per-file scan → `SourceCapability`
  headers, dtypes, cardinality, samples
- Task-level aggregation → `ExecutionProfile`
  - `context_size`, `source_shape`
  - `verifiability`, `operation_complexity`
  - `semantic_rule_required`
- Single source of truth — reused by prompt, static checker, guard, judge

Router rules (hard, deterministic):

- `weak` / `image` → multi_agent
- `long_docs` → RAG-enabled operator
- `semantic_rule_required` → tool_first_mixed
- `small+programmatic+low` → easy

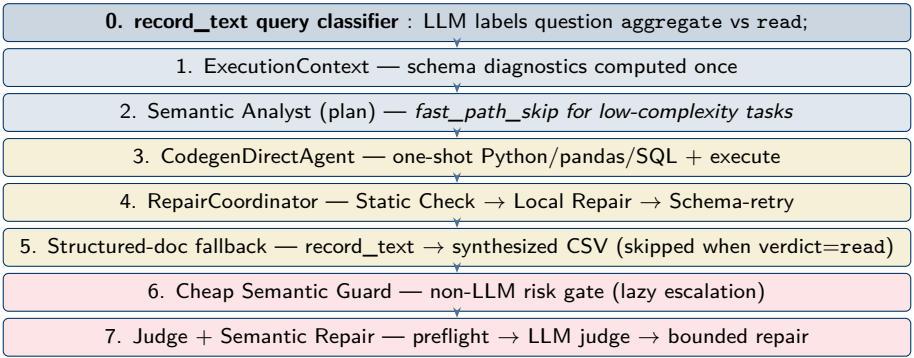
Cascade by *failure type*, not difficulty:

zero_rows → no upgrade (column wrong, not route)
retrieval_empty → RAG route
semantic_consistency_failed → tool_first_mixed
syntax/static/exec error → sibling operator

Cross-model verify (optional)

Primary + verifiers run in parallel; keep columns whose signatures agree $\geq$ `min_agreement` — directly targets the $-\lambda\cdot$ extra penalty.

Operator Pipeline — 8 Stages



Shared evidence — debug_steps

Every generated program emits `schema_inspection`, `used_columns`, `filter_conditions`, `join_keys`, `intermediate_counts`. Read by 3 consumers: cheap guard, preflight, LLM judge — single audit trail, no drift.

Schema Grounding + Static Checker + Local Repair

Schema Grounding

- Extract question concepts
- Match to real columns: Levenshtein + value match + cardinality
- Top-3 candidates w/ samples
- Fights hallucination

Static Checker

- Python AST + SQL scan
- `no_such_column`, `bad_join_key`, `_x`/`_y` suffixes
- Runs *before* execution
- Blocks predictable failures

Local Repair (0 LLM)

- 10+ deterministic patches
- Fuzzy column-name fix
- Cast merge keys to string
- Append missing `answer`
- Zero-row filter normalize

Why these three work together

All three read the **same** `SourceCapability` scan. The prompt tells the LLM what's real; the static checker verifies the LLM didn't invent; local repair fixes the remaining deterministic mistakes without an extra LLM call.

Cheap Semantic Guard — Non-LLM Risk Gate

Position: codegen succeeded, answer structurally valid, no analyst ran yet → trust this answer?

$$\text{risk\_score} = \min\left(1.0, \sum \text{unique\_weights}\right), \quad \text{escalate} = \text{any error} \vee \text{score} \geq 0.5$$

Category	Risk Code	Wt	Meaning
Execution	`execution_failed / empty_answer_rows`	1.0 / 0.75	No/zero answer rows
Repair	`suspicious_fallback:schema_retry`	0.8	Needed schema retry
Trace	`trace_missing:schema_inspection`	0.8	No real-column check
Trace	`zero_intermediate_count`	0.8	Mid-pipeline drops to 0
Coverage	`field_coverage_mismatch`	**0.75**	**Grounding candidate unused**
Formula	`formula_without_trigger`	0.75	Unasked formula used

Killer example

Q: *"patients with severe degree of thrombosis"*. Code uses `disease=="thrombosis"` but grounding found `degree_of_thrombosis` → `field_coverage_mismatch` (0.75) → escalate to judge instead of shipping wrong answer.

Lazy Escalation + Analyst-Exception Recovery

Decision flow:

- Codegen OK, answer valid
- Guard score ≥ 0.5?
  - **No** → fast return (0 extra LLM)
  - **Yes** → `plan(force=True)`
- Plan OK? Yes → Judge + Repair (max 3 rounds)
- Plan failed → fallback plan from guard risks → Judge

Observability — `final_gate` values:

- `plan+judge` — strong path
- `escalated+judge` — recovered
- `cheap_guard_only` — investigate
- `bypassed` — no semantic check

Analyst-exception recovery:

If analyst raises during plan, we used to fall through to a silently-skipped judge. Now:

- `stage==analyst_exception` → retry once with `force=True`
- Still fails → low-confidence synthetic plan; judge must run

Effect

Simple tasks: **1 LLM call total** (codegen only).

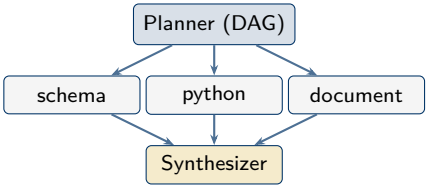
Hard tasks where analyst fails: guaranteed judge engagement instead of silent skip.

RAG Pipeline & Multi-Agent Orchestrator

RAG (long_docs route):

- Heading-aware chunking (1200 chars, overlap 150)
- **Query expansion**: paraphrases + HyDE
- **Hybrid retrieval**: BM25 || Embedding
- **RRF fusion** (rank-only, no score calibration)
- Optional cross-encoder reranker
- Embedding cache: `SHA256(model, text)`

Multi-agent (extreme route):



Specialists only see `report`; only Synthesizer can call `answer`. Same-layer specialists run in parallel.

Self-Consistency, Budget & Demo

Self-consistency — column-signature voting:

- N passes at T=0.7 with different seeds
- `target_count` = mode of column counts
- Top-`target_count` signatures by frequency = winning set
- Pick sample with max coverage, min extras
- Directly targets recall $- \lambda \cdot$ extra/pred

Budget controller (per task):

- LLM / tool / time caps
- Local & reasoner repair caps
- Multi-agent fallback cap

Streamlit demo:

- Real-time subprocess log (ProgressLogger)
- **Reasoning timeline**: schema $\rightarrow$ route $\rightarrow$ codegen $\rightarrow$ guard $\rightarrow$ judge
- Final answer table + generated Python
- Full `trace.json` (collapsible)

```
uv run streamlit run \
  scripts/demo_app.py
```

Auditable, explainable, reproducible.

Summary & Future Work

What we built:

- Tool-first Data Agent pipeline
- Deterministic task profiling (zero LLM)
- Shape-based routing + failure-type cascade
- 10+ local repair rules (zero LLM)
- Cheap Semantic Guard — non-LLM risk gate
- Analyst-exception recovery + audit trail
- Production-grade RAG (BM25+Emb+RRF)
- Multi-agent DAG with bounded replan
- Column-signature self-consistency

Observed performance (public):

Easy: ∼70–80 Medium/Hard: ∼60

Future work:

- Vision executor for image tasks
- Guard-weight auto-calibration from trace FP/FN
- Per-value majority voting within columns
- Smarter record-text extraction for long prose
- Submit to hidden test after tuning

Thank You

Questions?

Code: github.com/zuelmaruixin/kdd